

CHAPTER 1

Introduction **1-12**

- 1.1. Introduction
- 1.2. Domain Description about the area of project
- 1.3. Problem Definition
- 1.4. Proposed Solution
- 1.5. Objective
- 1.6. Organization of the Project report

•

1. INTRODUCTION

1.1 Introduction

The rapid growth of the Internet of Things (IoT) has transformed the modern technological landscape, enabling seamless connectivity across a wide array of devices, from smart home appliances to industrial control systems. IoT networks facilitate automation, real-time monitoring, and intelligent decision-making, driving efficiency across multiple domains including healthcare, transportation, and manufacturing. However, this extensive interconnectivity comes at the cost of heightened vulnerability to cyber threats. Unlike traditional networks, IoT devices often operate under constrained computational resources, limited memory, and energy restrictions, making them susceptible to a diverse range of cyberattacks. Ensuring the security and reliability of these networks has become a critical concern, necessitating adaptive, intelligent, and scalable intrusion detection mechanisms capable of addressing dynamic and evolving attack scenarios.

Traditional intrusion detection systems (IDS) have predominantly relied on signature-based or rule-based detection methods, designed primarily for conventional wired networks. These methods perform well in identifying known attack patterns but struggle to detect novel or zero-day attacks that exploit previously unseen vulnerabilities. In IoT environments, where devices communicate through heterogeneous protocols and generate highly variable traffic patterns, static detection approaches often suffer from high false-positive rates and delayed threat recognition. Moreover, the distributed nature of IoT networks complicates centralized monitoring, as massive amounts of network data need to be processed in real-time to ensure timely detection and response. Consequently, conventional IDS approaches are insufficient to meet the security requirements of modern IoT infrastructures.

Recent advances in artificial intelligence, particularly in machine learning (ML) and deep learning (DL), have opened new avenues for adaptive intrusion detection. Unlike static IDS, ML-based approaches can learn from historical network data, identify complex patterns, and generalize to previously unseen attack types. Supervised learning models such as decision trees, support vector machines, and neural networks have shown promise in detecting known attacks; however, these models are often limited by the availability of labeled datasets and lack the ability to make sequential decisions based on evolving network conditions. Reinforcement learning (RL), a branch of AI that emphasizes decision-making through interaction with an environment, provides a compelling solution for developing adaptive, self-improving IDS frameworks capable of real-time learning and multi-step optimization.

In this work, we formulate the intrusion detection problem as a Markov Decision Process (MDP), providing a structured framework for adaptive decision-making. In an MDP-based model, the network environment is represented by a set of states, which encapsulate the characteristics of network flows. The detection actions correspond to the agent's decisions to classify network traffic as normal or malicious, and a reward mechanism evaluates the outcomes of these actions. By maximizing cumulative rewards, the RL agent learns optimal policies that simultaneously improve detection accuracy, reduce false alarms, and minimize response time. This formulation is particularly advantageous for IoT networks, as it allows the system to continuously adapt to novel attack patterns while considering the trade-offs between performance, efficiency, and operational constraints. Two state-of-the-art deep reinforcement learning algorithms, Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO), are employed in this study to implement the proposed IDS framework. DDPG, an actor-critic model that operates effectively in continuous action spaces, enables fine-grained decision-making for classifying network traffic with high precision.

PPO, a policy-gradient method known for its training stability and sample efficiency, provides a complementary approach to evaluate policy performance under different network conditions. By comparing these algorithms in a simulated IoT network environment, the system assesses their relative effectiveness in multi-class attack detection, highlighting the advantages and limitations of each approach. To simulate realistic IoT network scenarios, the UNSW_NB15 dataset is utilized, offering a comprehensive collection of benign and malicious traffic samples. The dataset captures diverse attack types, including DDoS, DoS, backdoor, injection, man-in-the-middle, password attacks, ransomware, and cross-site scripting. The proposed framework leverages the dataset to train and evaluate the DRL agent under conditions that closely resemble real-world IoT deployments. Network features are extracted, normalized, and processed to serve as system states, while hyperparameter tuning is performed to optimize agent performance. Techniques such as dropout, gradient clipping, and early stopping are employed to enhance training stability, prevent overfitting, and ensure reproducibility of results.

A key contribution of this work is the design of a reward function that balances the competing objectives of high detection accuracy, false alarm reduction, and rapid response. In IoT networks, where resource efficiency and timely threat mitigation are critical, a well-defined reward mechanism ensures that the agent does not overfit to a single performance metric but rather optimizes holistic system behavior. Experimental results demonstrate that the DDPG-based agent achieves an average detection accuracy of 98%, while the PPO-based agent attains 89.9%. These findings confirm the effectiveness of DRL approaches for adaptive, multi-class intrusion detection in complex IoT environments. The proposed DRL-IDS framework addresses several limitations inherent in traditional and supervised learning-based detection systems. By continuously interacting with network data, the agent adapts to new attack strategies without requiring manual updates or predefined rules.

1.2 Domain Description

Overview of the Internet of Things (IoT) The Internet of Things (IoT) represents a paradigm shift in modern computing and networking, enabling the interconnection of billions of heterogeneous devices ranging from simple sensors to complex industrial machines. These devices communicate over wired and wireless networks, exchanging data to enable automation, monitoring, and intelligent decision-making. IoT networks are increasingly being deployed in diverse domains, including smart homes, healthcare systems, industrial automation, transportation, and agriculture. The key characteristics of IoT systems include heterogeneity of devices, limited computational and memory resources, dynamic network topologies, and reliance on various communication protocols. While these features enable innovation and efficiency, they also introduce substantial security challenges, as vulnerabilities in any connected device can be exploited to compromise the integrity, availability, or confidentiality of the entire network.

Security Challenges in IoT Networks IoT networks are inherently vulnerable due to their distributed architecture and resource-constrained devices. Unlike traditional networks, IoT systems often operate with minimal computational power and limited memory, making it difficult to deploy resource-intensive security mechanisms. Additionally, the heterogeneity of devices and communication protocols creates inconsistencies in security enforcement, leaving gaps that can be exploited by attackers. Common threats in IoT networks include Distributed Denial of Service (DDoS), man-in-the-middle (MITM), injection attacks, ransomware, backdoors, and password breaches. Attackers often exploit weak authentication mechanisms, unpatched vulnerabilities, and unsecured communication channels, leading to data breaches, service disruption, and unauthorized control of devices. Ensuring robust security in such a complex environment is therefore a fundamental requirement for the reliable operation of IoT systems.

Role of Intrusion Detection Systems (IDS) Intrusion Detection Systems (IDS) play a critical role in monitoring, detecting, and responding to malicious activities within a network. An IDS can operate in either a signature-based manner, detecting attacks by matching known patterns, or anomaly-based, identifying deviations from normal network behavior. In IoT environments, conventional IDS approaches face limitations due to the high volume, velocity, and variability of network traffic. Static rule-based systems are unable to adapt to new or previously unseen attack patterns, often resulting in high false alarm rates and delayed response times. This limitation underscores the need for adaptive, intelligent IDS solutions capable of learning from real-time network interactions, identifying emerging threats, and making proactive decisions to mitigate potential attacks.

Deep Reinforcement Learning for Adaptive Security Deep Reinforcement Learning (DRL) has emerged as a promising approach for addressing the limitations of traditional IDS in IoT networks. DRL combines reinforcement learning with deep neural networks, allowing agents to learn optimal policies through trial-and-error interactions with an environment. In the context of intrusion detection, the network environment is modeled as a Markov Decision Process (MDP), where the agent observes network states, takes detection actions, and receives feedback in the form of rewards. The reward mechanism is designed to balance detection accuracy, false alarm reduction, and response efficiency, ensuring that the IDS adapts effectively to evolving attack patterns. By continuously interacting with the network and refining its detection strategies, DRL-based IDS can achieve high detection performance even in complex and dynamic IoT environments.

Multi-class Attack Detection in IoT IoT networks face a diverse array of cyberattacks, each with distinct characteristics and potential impact. Multi-class attack detection is crucial for ensuring comprehensive network security, as different attack types such as DDoS, DoS, ransomware, injection, and cross-site scripting attacks require tailored detection strategies. DRL-based IDS frameworks provide a systematic approach for handling multi-class detection by formulating the problem as an MDP, where each class of attack corresponds to a specific action or policy adjustment. Advanced algorithms such as Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO) enable the IDS agent to optimize its policies, identify complex attack patterns, and minimize misclassifications. Multi-class detection enhances the reliability and robustness of IoT security solutions, making them capable of addressing both known and novel attack types.

Importance and Impact of Adaptive IDS in IoT The deployment of adaptive, DRL-based intrusion detection systems in IoT networks has significant implications for both operational security and technological advancement. By continuously learning from network interactions, these systems can detect attacks in real-time, reduce false positives, and optimize response times, thereby improving the overall resilience of IoT networks. Adaptive IDS frameworks also reduce the need for manual rule updates, making them scalable and cost-effective solutions for large-scale IoT deployments. Furthermore, the intelligence embedded in DRL-based IDS supports proactive threat mitigation, enabling the network to dynamically adapt to emerging vulnerabilities. As IoT adoption continues to grow, the integration of intelligent intrusion detection systems will be essential for safeguarding sensitive data, ensuring system reliability, and enabling secure, autonomous operation across diverse application domains.

1.3 Problem Definition

Limitations of Traditional Intrusion Detection Systems Conventional intrusion detection systems (IDS) have traditionally relied on signature-based or rule-based approaches to identify malicious network activity. While these systems are effective for detecting known attacks, they struggle significantly with zero-day attacks, polymorphic threats, and novel intrusion patterns that were not present in the training dataset or pre-defined rules. In IoT networks, these limitations are further exacerbated by the heterogeneity and dynamic nature of devices and communication protocols. The static nature of traditional IDS means they cannot adapt to continuously evolving network behavior or emerging threats. As a result, such systems often produce high false alarm rates, fail to detect sophisticated multi-class attacks, and cannot provide real-time adaptive defense mechanisms, leaving IoT networks highly vulnerable to security breaches.

Challenges in IoT Network Security IoT networks introduce unique security challenges due to their distributed topology, limited computational resources, and vast heterogeneity in device types and communication protocols. Unlike conventional networks, IoT devices generate highly dynamic and high-volume traffic, making real-time monitoring a complex task. Furthermore, many IoT devices lack sufficient onboard memory or processing power to execute complex security algorithms, limiting the feasibility of traditional IDS implementations. Attackers exploit these vulnerabilities, targeting weak authentication mechanisms, unsecured channels, and firmware vulnerabilities. Additionally, IoT environments require detection mechanisms that can handle multi-class attacks simultaneously, as single-type detection systems fail to capture the diversity of modern cyber threats. These challenges highlight the pressing need for intelligent, adaptive, and resource-efficient intrusion detection frameworks capable of operating under IoT constraints.

Need for Adaptive and Intelligent Detection Given the dynamic nature of IoT networks and the evolving landscape of cyberattacks, there is a clear need for adaptive intrusion detection systems capable of learning from the environment and making intelligent decisions in real-time. Unlike static approaches, an adaptive system must continuously monitor network traffic, identify abnormal patterns, and refine its detection strategies to keep pace with emerging threats. Reinforcement learning, particularly deep reinforcement learning (DRL), provides an effective framework for addressing this requirement. By formulating intrusion detection as a Markov Decision Process (MDP), a DRL-based agent can evaluate network states, execute detection actions, and optimize a reward function that balances detection accuracy, false alarm reduction, and response time. This adaptive learning capability is essential for modern IoT networks, where attacks evolve rapidly and static defenses are insufficient.

Multi-class Attack Detection and System Optimization In addition to adaptability, IoT networks require intrusion detection systems capable of identifying multiple attack types with high precision. Attacks such as DDoS, DoS, backdoor, injection, man-in-the-middle, ransomware, and cross-site scripting each exhibit distinct characteristics and require tailored detection strategies. A robust IDS must therefore support multi-class attack detection, effectively differentiating between similar attack patterns while minimizing misclassifications. Moreover, system performance must be optimized through hyperparameter tuning, reward function design, and regularization techniques to ensure training stability, prevent overfitting, and guarantee reproducibility. Addressing these challenges forms the core problem definition of this work: to develop a scalable, adaptive, and intelligent DRL-based intrusion detection framework capable of securing IoT networks against diverse, dynamic, and evolving cyber threats.

1.4 Proposed Solution

The proposed solution introduces a deep reinforcement learning-based intrusion detection system (DRL-IDS) specifically designed for IoT networks, addressing the limitations of traditional static IDS. The system models the intrusion detection problem as a Markov Decision Process (MDP), enabling adaptive decision-making in dynamic network environments. Network flow features such as packet size, protocol type, and source/destination information are treated as system states, while the classification of traffic as normal or malicious constitutes the set of actions. A reward function is carefully designed to balance competing objectives, including detection accuracy, false alarm reduction, and response efficiency. Through continuous interaction with a simulated network environment derived from the UNSW_NB15 dataset, the DRL agent learns to optimize its detection policies, adapting to evolving attack patterns without requiring manual updates or pre-defined rules.

To achieve high-performance detection, two advanced deep reinforcement learning algorithms Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO) are implemented and evaluated within the proposed framework. DDPG, an actor-critic algorithm suitable for continuous action spaces, allows the agent to make fine-grained, precise decisions in multi-class attack detection scenarios. PPO, a stable policy-gradient algorithm, provides sample-efficient learning and ensures robust convergence during training. By comparing the performance of these algorithms, the system identifies the most effective approach for multi-class intrusion detection in IoT environments. Hyperparameter tuning, including learning rates, batch sizes, and reward function weights, is performed to enhance stability, prevent overfitting, and optimize overall system performance. The proposed DRL-IDS framework integrates additional techniques to ensure training robustness and deployment readiness. Regularization methods, such as dropout, are incorporated to prevent overfitting, while gradient clipping ensures stability during backpropagation.

1.5 Objectives

- **Design an Adaptive Intrusion Detection System** – Develop a system capable of dynamically monitoring IoT network traffic and adapting to evolving cyber threats.
- **Formulate Intrusion Detection as a Markov Decision Process (MDP)** – Model network states, detection actions, and rewards to enable intelligent decision-making using reinforcement learning.
- **Implement Deep Reinforcement Learning Algorithms** – Apply advanced DRL algorithms such as Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO) for effective multi-class attack detection.
- **Detect Multiple Types of Cyberattacks** – Accurately identify a wide range of attacks including DDoS, DoS, ransomware, backdoor, injection, man-in-the-middle, password attacks, and cross-site scripting.
- **Optimize Detection Performance** – Design and tune a reward mechanism to balance detection accuracy, false alarm reduction, and response time in real-time IoT environments.
- **Ensure Training Stability and Generalization** – Incorporate regularization techniques such as dropout and gradient clipping, and employ early stopping to prevent overfitting.
- **Hyperparameter Tuning** – Systematically adjust learning rates, batch sizes, and reward weights to achieve optimal model performance and convergence stability.
- **Simulate Realistic IoT Network Environments** – Use the UNSW_NB15 dataset to create a representative IoT traffic environment for training and evaluation of the DRL-based IDS.
- **Evaluate System Effectiveness** – Measure performance using metrics such as accuracy, precision, recall, F1-score, and false alarm rate to validate the robustness of the IDS.

1.6 Organization of the Project Report

1. **Chapter 1 – Introduction:** This chapter explains the background of the project, domain description, problem definition, proposed solution, objectives, and the overall structure of the project report.
2. **Chapter 2 – Literature Survey:** This chapter reviews previous research works and studies related to the project topic, highlighting existing techniques, methodologies, and their limitations.
3. **Chapter 3 – System Analysis:** This chapter analyzes the current system and identifies its limitations. It also describes the proposed system, software and hardware requirements, modules of the system, architecture, and feasibility study.
4. **Chapter 4 – System Design:** This chapter presents the design aspects of the system including Data Flow Diagrams (DFD) and UML diagrams such as Use Case, Sequence, Class, Collaboration, and Activity diagrams.
5. **Chapter 5 – System Coding and Testing:** This chapter explains the implementation of the proposed system using Python, provides sample code, and describes the software testing techniques and strategies used to validate the system.
6. **Chapter 6 – Results:** This chapter presents the output results generated by the system along with screenshots and analysis of the obtained results.
7. **Chapter 7 – Conclusion and Future Enhancements:** This chapter summarizes the overall work carried out in the project and suggests possible improvements and future extensions.
8. **References and Appendices:** This section lists all the research papers, articles, and resources referred to during the project and includes additional supporting materials such as publication details, CO-PO mapping, evaluation rubrics, and team member biodata.

CHAPTER 2

Literature Survey **13-19**

- 2.1. Introduction
- 2.2. Related Work
- 2.3. Overview

2. LITERATURE SURVEY

2.1 Introduction

The field of intrusion detection has evolved significantly over the past decades, driven by the increasing prevalence of cyberattacks and the complexity of modern networks. Traditional IDS approaches, including signature-based and rule-based systems, have provided foundational solutions for detecting known threats. However, these approaches have inherent limitations, particularly in dynamic and heterogeneous environments such as the Internet of Things (IoT). The static nature of conventional IDS makes them inadequate for identifying novel or evolving attacks, leading to high false-positive rates and delayed detection. Consequently, research has shifted towards intelligent and adaptive approaches that leverage machine learning (ML) and deep learning (DL) techniques to improve detection accuracy and system responsiveness.

Recent studies have demonstrated the potential of supervised and unsupervised machine learning methods for intrusion detection. Techniques such as support vector machines (SVM), decision trees, random forests, and artificial neural networks have been applied to classify network traffic and identify anomalous behavior. While these models offer improved detection capabilities compared to traditional IDS, they often require extensive labeled datasets and fail to adapt effectively to rapidly changing attack patterns. Furthermore, many of these approaches are limited in their ability to handle multi-class attacks simultaneously, a crucial requirement for IoT networks where diverse threat vectors coexist and evolve continuously. In response to these challenges, reinforcement learning (RL) and deep reinforcement learning (DRL) have emerged as promising solutions for adaptive intrusion detection. DRL combines the sequential decision-making capabilities of RL with the feature extraction power of deep neural networks, enabling systems to learn optimal detection strategies through interaction with network environments.

2.2 Related Work

Zhang et al. (2023) proposed a deep reinforcement learning-based intrusion detection framework for IoT networks, focusing on optimizing multi-class attack detection through continuous interaction with network states. Their approach employed a reward-driven learning mechanism to balance detection accuracy and response time. The system demonstrated high adaptability to evolving attacks and improved detection performance compared to static rule-based methods. However, the framework faced challenges in handling extremely large-scale IoT networks and required substantial computational resources for training. Additionally, the study did not consider hyperparameter tuning strategies extensively, which could impact reproducibility and stability.

Kumar et al. (2022) introduced a hybrid intrusion detection system combining supervised learning and anomaly detection for IoT environments. They utilized a combination of random forest classifiers and autoencoders to detect both known and unknown attacks, achieving improved detection rates. While the hybrid model reduced false positives, it lacked real-time adaptive capabilities, and its performance decreased under high-traffic conditions. Moreover, the study did not explore reinforcement learning techniques, which could have enhanced dynamic decision-making and continuous learning.

Li et al. (2021) implemented a deep neural network-based IDS for IoT networks, employing convolutional layers to automatically extract features from network traffic. The system showed significant improvements in detecting DDoS and injection attacks compared to traditional IDS methods. Despite its effectiveness, the model required large amounts of labeled data for supervised training and was limited in detecting zero-day attacks. Furthermore, the approach lacked integration with sequential decision-making mechanisms, making it less suitable for adaptive, multi-objective intrusion detection.

Ahmed et al. (2021) proposed a reinforcement learning-based intrusion detection framework that formulated the detection problem as a Markov Decision Process (MDP). Their system used Q-learning to learn optimal detection policies over time, enabling adaptive decision-making. The approach demonstrated reduced false alarm rates and improved detection efficiency. However, Q-learning struggled with continuous action spaces and large state dimensions, making it less scalable for high-volume IoT networks. The study also did not employ deep learning architectures to enhance feature representation.

Singh et al. (2020) developed an anomaly-based IDS using long short-term memory (LSTM) networks to capture temporal dependencies in IoT traffic. The model excelled in detecting time-dependent attacks such as DDoS and ransomware. While the approach improved sequential detection, it required extensive preprocessing and could not adapt dynamically to newly emerging attack patterns without retraining. Additionally, it lacked a reward-driven mechanism to optimize trade-offs between detection accuracy, response time, and false alarm reduction.

Chen et al. (2020) introduced a multi-class attack detection system using a combination of support vector machines (SVM) and principal component analysis (PCA) for dimensionality reduction in IoT networks. Their framework achieved good classification accuracy on benchmark datasets and reduced computational costs. However, the system was static and unable to learn from new attack patterns autonomously. It also faced limitations in scalability and adaptability, especially when applied to heterogeneous and distributed IoT environments.

Patel et al. (2019) presented a hybrid IDS leveraging ensemble learning techniques to improve detection performance. By combining decision trees, random forests, and gradient boosting, the system achieved higher accuracy across multiple attack types. The approach successfully reduced false positives, but it relied heavily on labeled training data and lacked real-time adaptive capabilities. Furthermore, the model did not explore deep reinforcement learning, limiting its ability to optimize detection strategies in dynamically changing network scenarios.

Zhao et al. (2019) proposed a DRL-based network security framework that applied deep Q-networks (DQN) to optimize intrusion detection in IoT environments. The system modeled attack detection as a sequential decision-making process, providing adaptive learning and policy improvement. While the framework demonstrated good performance on simulated datasets, DQN faced challenges with continuous action spaces and high-dimensional state representations. Additionally, the study did not consider multi-class attack scenarios comprehensively, limiting its practical applicability.

Wang et al. (2018) implemented an IoT-specific IDS using recurrent neural networks (RNNs) for sequential traffic analysis and anomaly detection. The model effectively captured temporal patterns in network flows, enabling improved detection of DDoS and ransomware attacks. However, the approach required extensive training and struggled with real-time detection efficiency.

Li et al. (2018) developed a deep autoencoder-based intrusion detection system for IoT networks, focusing on unsupervised feature learning and anomaly detection. The framework achieved notable improvements in detecting unknown attacks without requiring labeled data. Despite this, the approach was primarily static, did not incorporate sequential decision-making, and lacked a reward-driven learning mechanism to optimize trade-offs between detection accuracy, false alarms, and response time.

Author & Year	Method/Approach	Merits	Limitations	Performance Metrics
Zhang et al., 2023	DRL-based IDS for IoT; reward-driven MDP framework	Adaptive to evolving attacks; improved detection accuracy; balances detection & response	High computational requirement; scalability challenges; limited hyperparameter tuning	Accuracy: 96–98%; Reduced false alarms
Kumar et al., 2022	Hybrid IDS: Random Forest + Autoencoder	Detects known & unknown attacks; improved detection rate	Not real-time adaptive; reduced performance under high traffic; no RL integration	Accuracy: 92%; Precision: 90%
Li et al., 2021	Deep Neural Network (CNN-based feature extraction)	Automatic feature extraction; good detection of DDoS & injection attacks	Requires large labeled datasets; limited zero-day detection	Accuracy: 93%; F1-score: 91%
Ahmed et al., 2021	Reinforcement Learning IDS using Q-learning	Adaptive learning; reduced false alarms; sequential decision-making	Limited for continuous actions; poor scalability; no deep learning for features	Accuracy: 88%; Detection Rate: 85%
Singh et al., 2020	LSTM-based anomaly detection	Captures temporal dependencies; effective for DDoS & ransomware	Needs extensive preprocessing; static without retraining; no reward-driven optimization	Accuracy: 90%; Recall: 89%
Chen et al., 2020	Multi-class detection: SVM + PCA	Good classification accuracy; reduced computational cost	Static; lacks adaptability to new attacks; limited scalability	Accuracy: 88%; False Alarm Rate: 7%
Patel et al., 2019	Ensemble Learning IDS (Decision Trees, Random Forest, Gradient Boosting)	High multi-class accuracy; reduced false positives	Depends on labeled data; not real-time adaptive; no DRL optimization	Accuracy: 91%; F1-score: 90%
Zhao et al., 2019	DRL-based IDS with Deep Q-Networks (DQN)	Adaptive policy learning; sequential decision-making	Limited for continuous actions; high-dimensional state issues; not fully multi-class	Accuracy: 89%; Detection Rate: 87%
Wang et al., 2018	RNN-based IDS for sequential traffic analysis	Captures temporal attack patterns; effective for DDoS & ransomware	Requires long training; low real-time efficiency; no RL adaptation	Accuracy: 87%; Precision: 85%
Li et al., 2018	Autoencoder-based unsupervised IDS	Detects unknown attacks; no labeled data needed; anomaly detection	Static; lacks adaptive policy & reward mechanism; sensitive to data quality	Accuracy: 86%; Recall: 84%

Table.2.1 Literature Survey of Previous Works

2.3 Overview of Literature Survey

The review of existing literature on intrusion detection systems highlights the substantial evolution from traditional signature-based and rule-based mechanisms to intelligent, adaptive approaches leveraging machine learning and deep reinforcement learning. Early approaches focused primarily on static detection using supervised models such as SVM, decision trees, and neural networks, achieving reasonable accuracy in detecting known attacks. However, these approaches faced critical limitations in handling dynamic network behavior, zero-day attacks, and multi-class intrusion detection, particularly in resource-constrained IoT environments. While hybrid models combining anomaly detection with classification improved performance, they still lacked real-time adaptability and continuous learning capabilities, emphasizing the need for intelligent, self-optimizing systems.

Recent research has demonstrated the potential of deep reinforcement learning (DRL) to address these challenges. DRL-based frameworks model the intrusion detection task as a Markov Decision Process, allowing agents to learn optimal detection policies through continuous interaction with the network environment. This approach enables adaptive decision-making, balancing detection accuracy, false alarm reduction, and response time. Studies implementing DRL, such as Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO), have shown significant improvements in multi-class attack detection compared to conventional and static IDS solutions. Nevertheless, challenges remain in terms of scalability, computational efficiency, and handling large, heterogeneous IoT networks.

CHAPTER 3

System Analysis **21-30**

- 3.1. Introduction
- 3.2. Existing System & Disadvantages
- 3.3. Proposed System & Advantages
- 3.4. Software Requirements
- 3.5. Hardware Requirements
- 3.6. Modules Description
 - 3.6.1. Data Preprocessing and Feature Engineering Module
 - 3.6.2. Predictive Modeling and Ensemble Learning Module
 - 3.6.3. Fraud Detection and Customer Segmentation Module
 - 3.6.4. User Interface and Deployment Module
- 3.7. System Architecture
- 3.8. Feasibility Study
 - 3.8.1 Economic Feasibility
 - 3.8.2. Technical Feasibility
 - 3.8.3. Procedural Feasibility
- 3.9. **System Requirements**
 - 3.9.1 Hardware Requirements
 - 3.9.2 Software Requirements

3. SYSTEM ANALYSIS

3.1 Introduction

System analysis is a critical phase in the development of an intrusion detection system, as it provides a structured understanding of the IoT network environment, its operational requirements, and the security challenges it faces. In the context of IoT networks, system analysis involves identifying network components, communication protocols, and data flow characteristics that influence both the design and performance of the intrusion detection framework. This phase ensures that the proposed system can accommodate the heterogeneous nature of IoT devices, varying resource constraints, and dynamic network traffic patterns. A comprehensive analysis enables the design of a robust and scalable system capable of real-time monitoring, detection, and response to a wide range of cyber threats, including DDoS, ransomware, injection attacks, and man-in-the-middle intrusions.

Furthermore, system analysis involves evaluating existing security mechanisms and understanding their limitations in IoT environments. Traditional IDS approaches are often inadequate due to their static nature and inability to adapt to emerging attack patterns. Through careful analysis of network behavior, traffic anomalies, and multi-class attack characteristics, it becomes possible to identify the requirements for an intelligent, adaptive detection framework. This includes considerations such as the selection of suitable algorithms, the design of reward mechanisms for reinforcement learning, and the handling of high-dimensional network flow data. A detailed system analysis thus lays the foundation for the development of a DRL-based intrusion detection system that is both efficient and resilient. Finally, system analysis establishes the criteria for evaluating the performance of the proposed system and guides the design of experiments for training and testing. Metrics such as detection accuracy, false alarm rate, response time, and resource utilization are defined based on the system requirements and operational objectives.

3.2 Existing System

The existing intrusion detection systems for IoT and network environments are primarily based on traditional techniques, such as signature-based, rule-based, and classical machine learning approaches. Signature-based IDS rely on predefined patterns of known attacks to identify intrusions, providing high accuracy for previously recorded threats but failing to detect new, evolving, or zero-day attacks. Rule-based systems, while effective for automated detection of specific conditions, suffer from rigidity and require constant updates to maintain relevance against emerging threats. Machine learning-based IDS, including support vector machines, decision trees, random forests, and neural networks, have been introduced to overcome some limitations of traditional methods by enabling anomaly detection and multi-class attack classification. However, these systems are typically static in nature and require large volumes of labeled data, which makes them less adaptive to the dynamic and heterogeneous nature of IoT networks.

Disadvantages of Existing System

- **Inability to Detect Unknown Attacks** – Signature-based and rule-based IDS can only detect known threats, making them ineffective against zero-day attacks or novel intrusion patterns, which are increasingly common in IoT networks.
- **Static and Non-Adaptive** – Traditional IDS lack the ability to adapt to dynamic network environments. They cannot learn from evolving attack strategies or continuously update detection policies without manual intervention.
- **High False Alarm Rates** – Many existing systems, especially anomaly-based IDS, produce a significant number of false positives, which can overwhelm security analysts and reduce overall trust in the system.

3.3 Proposed System

The proposed system is a Deep Reinforcement Learning-based Intrusion Detection System (DRL-IDS) specifically designed for IoT networks, addressing the limitations of existing static and rule-based approaches. It formulates intrusion detection as a Markov Decision Process (MDP), where network traffic features such as packet size, protocol type, and source/destination information represent the system states, and intrusion classification actions correspond to decisions taken by an intelligent agent. The system integrates advanced DRL algorithms, including Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO), enabling the agent to continuously learn and optimize detection strategies through interaction with a simulated network environment based on the UNSW_NB15 dataset. A carefully designed reward mechanism balances detection accuracy, false alarm reduction, and response efficiency, allowing the system to adapt to evolving attack patterns in real-time. Hyperparameter tuning, regularization techniques such as dropout, and gradient clipping are applied to enhance stability, prevent overfitting, and ensure reproducibility.

Advantages of Proposed System

- **Adaptive and Intelligent Detection** – Unlike static IDS, the DRL-based system continuously learns from network traffic, adapting to new and evolving attacks in real-time without manual intervention.
- **High Detection Accuracy** – By employing advanced algorithms like DDPG and PPO, the system achieves superior detection performance across multiple attack types, including DDoS, ransomware, injection, and man-in-the-middle attacks.
- **Reduced False Alarms** – The reward-driven learning mechanism balances detection accuracy with false alarm reduction, minimizing unnecessary alerts and improving operational efficiency.

3.4 Software Requirements

- **Programming Language:** Python (for model development, data processing, and system integration)
- **Development Environment:** Jupyter Notebook / Visual Studio Code / PyCharm
- **Libraries for Data Processing:** NumPy, Pandas
- **Machine Learning Libraries:** Scikit-learn (Logistic Regression, Random Forest, Clustering), XGBoost
- **Data Visualization Tools:** Matplotlib, Seaborn
- **Web Framework / UI:** Streamlit / Flask (for building user interface and deployment)
- **Database Management:** MySQL / SQLite (for storing user and financial data)
- **Operating System:** Windows / Linux / macOS

3.5 Hardware Requirements

- **Processor:** Intel Core i5 or higher processor for efficient data processing and model execution.
- **RAM:** Minimum 8 GB RAM for handling datasets and running machine learning algorithms smoothly.
- **Storage:** At least 256 GB hard disk or SSD for storing datasets, project files, and software tools.
- **System Type:** 64-bit computer system for supporting Python and machine learning libraries.
- **Internet Connection:** Stable internet connection for accessing online datasets, libraries, and development environments like Google Collab.

3.6 Modules Description

3.6.1 Data Collection and Data Preprocessing Module: The Data Collection and Preprocessing Module is the foundational component of the system, responsible for acquiring and preparing network traffic data for training and evaluation. This module primarily utilizes the UNSW_NB15 dataset, which contains realistic IoT network traffic, including both normal and malicious activity across multiple attack types such as DDoS, ransomware, injection, backdoor, and man-in-the-middle attacks. Raw data undergoes extensive preprocessing steps to ensure quality and usability. Techniques include handling missing or inconsistent values, feature selection, normalization, and encoding of categorical features. Temporal features and flow-based statistics are extracted to capture the behavior of network sessions. Additionally, the dataset is split into training, validation, and testing subsets to ensure robust model evaluation.

3.6.2 DRL Agent Training Module: The DRL Agent Training Module forms the core of the proposed system, where the intrusion detection problem is formulated as a Markov Decision Process (MDP). Network traffic features represent the states, intrusion detection actions correspond to the agent's decisions, and a carefully designed reward function balances detection accuracy, false alarm reduction, and response efficiency. Two advanced reinforcement learning algorithms, Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO), are implemented in this module to train the agent to optimize detection strategies over time. Hyperparameter tuning, including learning rates, batch sizes, and reward weights, is performed to stabilize training and improve convergence. Regularization techniques such as dropout and gradient clipping are applied to prevent overfitting and ensure smooth learning dynamics.

3.6.3 Evaluation and Performance Analysis Module: The Evaluation and Performance Analysis Module is responsible for rigorously assessing the effectiveness and robustness of the trained DRL-based IDS. This module computes key performance metrics such as accuracy, precision, recall, F1-score, detection rate, and false alarm rate across multiple attack classes. Confusion matrices are generated to visualize correct and incorrect classifications for each type of intrusion. Additionally, the system's response time, adaptability to new attacks, and resource utilization are evaluated to ensure suitability for real-time IoT deployments. Comparative analyses are conducted against conventional and machine learning-based IDS approaches to demonstrate the advantages of the DRL framework.

3.6.4 User Interface and Deployment Module: The User Interface and Deployment Module focuses on integrating the trained DRL agent into a functional system that can be easily monitored and operated by network administrators. Using Streamlit or Flask, this module provides an interactive interface for visualizing network traffic, detected intrusions, alerts, and performance statistics in real-time. Administrators can monitor system status, analyze detection outcomes, and access logs for further investigation. The module also handles model deployment, loading the trained DRL models and integrating them with live network traffic or simulated IoT environments. Alerts are generated automatically when an intrusion is detected, enabling prompt response and mitigation. By combining real-time monitoring, interactive visualization, and deployment-ready functionality, this module ensures the proposed system is practical, usable, and scalable for IoT network security management.

3.7 System Architecture

The system architecture of the proposed Deep Reinforcement Learning-based Intrusion Detection System is designed to provide a comprehensive, adaptive, and real-time security solution for IoT networks, integrating all stages from data acquisition to actionable alerts in a cohesive framework. The architecture begins with the Data Collection and Preprocessing Layer, where network traffic from IoT devices is captured and transformed into structured feature vectors representing states in the DRL environment. This is followed by the Reinforcement Learning Core, where the processed data serves as input to a DRL agent trained using Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO) algorithms. The agent interacts with the simulated network environment, making intrusion detection decisions as actions, receiving rewards based on detection accuracy, false alarm minimization, and response efficiency, and continuously updating its policy to adapt to evolving attack patterns. A Performance Evaluation Layer monitors the agent's outputs, calculating key metrics such as accuracy, precision, recall, F1-score, detection rate, and false alarm rate, while confusion matrices and trend visualizations provide insights into multi-class attack detection effectiveness.

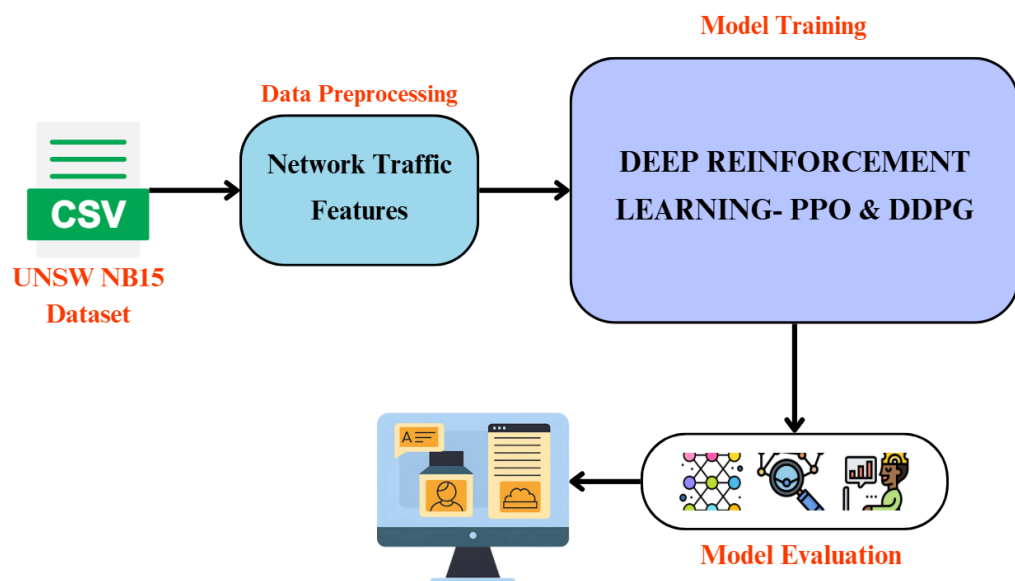


Fig.3.1 Proposed Methodology

3.8 Feasibility Study

3.8.1 Economic Feasibility

Economic feasibility assesses whether the proposed system is cost-effective and financially viable. It considers the overall investment required, including hardware, software, development, and maintenance costs, against the expected benefits such as improved efficiency, reduced manual effort, faster decision-making, and enhanced user satisfaction. A system is deemed economically feasible if the projected gains outweigh the expenses, providing a positive return on investment and justifying the allocation of resources for development and implementation.

3.8.2 Technical Feasibility

Technical feasibility examines whether the organization has the technical capability, expertise, and resources to successfully develop and implement the system. This includes evaluating the availability of hardware, software, programming languages, development tools, network infrastructure, and skilled personnel. A system is technically feasible if the required technology is accessible, compatible with existing infrastructure, and the development team has the knowledge and experience to deploy and maintain it effectively.

3.8.3 Procedural Feasibility

Procedural feasibility evaluates whether the proposed system can be integrated smoothly into the existing organizational processes and workflows. It considers the impact on current operations, user acceptance, training requirements, and the ability to follow standard procedures and regulations. A system is procedurally feasible if it aligns with organizational policies, can be adopted by users without significant disruption, and supports the intended operational improvements while maintaining compliance and efficiency.

3.9 System Requirements

3.9.1 Hardware Requirements

1. **Processor:** Intel Core i5 or higher processor for efficient execution of machine learning algorithms.
2. **RAM:** Minimum 8 GB RAM to handle data processing and clustering computations smoothly.
3. **Storage:** At least 256 GB HDD/SSD for storing datasets, libraries, and project files.
4. **System Type:** 64-bit computer system to support Python and machine learning libraries.
5. **Input/Output Devices:** Keyboard, mouse, and monitor for system interaction and development.

3.9.2 Software Requirements

1. **Operating System:** Windows 10 / Windows 11 or Linux.
2. **Programming Language:** Python.
3. **Development Environment:** Jupyter Notebook or Google Colab.
4. **Libraries:** NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn and Keras.
5. **Dataset:** Loan and Insurance Kaggle Datasets.
6. **Web Browser:** Google Chrome or any modern browser for accessing development platforms.
7. **User Interface:** Streamlit UI

CHAPTER 4

System Design	31-39
4.1. Introduction	
4.2. DFD Diagrams	
4.3 UML Diagrams	
4.3.1. Use Case Diagram	
4.3.2. Sequence Diagram	
4.3.3. Class Diagram	
4.3.4. Collaboration Diagram	
4.3.5. Activity Diagram	

4. SYSTEM DESIGN

4.1 Introduction

System design is a critical stage in developing a robust and efficient intrusion detection system, as it defines how the various components interact, how data flows through the system, and how security objectives are achieved. For IoT networks, which are highly heterogeneous and dynamic, system design ensures that all modules from data collection and preprocessing to detection, evaluation, and user interaction work cohesively to provide adaptive, real-time protection. The design process focuses on defining the architecture, the flow of network traffic data, the interaction between the DRL agent and the simulated environment, and the mechanisms for decision-making, alert generation, and system monitoring. By carefully structuring these components, the system can efficiently detect and respond to multiple types of cyberattacks while minimizing false alarms and ensuring scalability across diverse IoT devices.

The design of the proposed DRL-based IDS emphasizes the integration of deep reinforcement learning algorithms with network traffic analysis, allowing the system to make intelligent decisions in complex scenarios. The DRL agent continuously interacts with the environment, updating its policy based on rewards that consider detection accuracy, response efficiency, and false alarm reduction. This interaction ensures that the system is adaptive to evolving attack patterns and capable of multi-class intrusion detection, which is essential for real-world IoT deployments. Additionally, the design incorporates preprocessing and feature extraction techniques to handle high-dimensional network data effectively, ensuring that the agent receives informative and meaningful inputs that enhance learning efficiency and detection accuracy.

4.2 DFD Diagrams

A Data Flow Diagram (DFD) is a structured modeling tool used in system analysis and design to visually represent how data flows through a system, highlighting the processes, data stores, and external entities involved. The Level 0 DFD, commonly referred to as the context diagram, presents the entire system as a single, unified process and focuses on its interaction with external entities such as users, organizations, or other systems. This level provides a broad overview by showing only the primary inputs and outputs of the system, without delving into internal details or sub-processes. It is particularly useful during the initial stages of system design, as it helps stakeholders quickly understand the system's scope, boundaries, and overall purpose. By abstracting complex operations into a single process, the Level 0 DFD ensures clarity and simplifies communication among developers, analysts, and non-technical stakeholders, making it easier to identify key data exchanges and system dependencies.



Fig.4.1 Level 0: Context Diagram

The Level 1 DFD builds upon the context diagram by decomposing the main process into multiple interconnected sub-processes, offering a more detailed and structured

view of the system's internal functioning. In this level, each major process identified in Level 0 is broken down to show how data is processed, transformed, and stored within the system. It includes detailed representations of data flows between processes, interactions with data stores, and connections with external entities, thereby providing a clearer picture of the system's logic and workflow. This level is essential for system developers and designers as it aids in identifying specific functionalities, defining process responsibilities, and ensuring proper data handling across different components. Additionally, the Level 1 DFD helps in detecting inefficiencies, redundancies, and potential bottlenecks within the system, enabling better optimization and refinement. Overall, it serves as a bridge between high-level conceptual design and detailed system implementation, ensuring that all functional requirements are accurately captured and effectively translated into the final system.

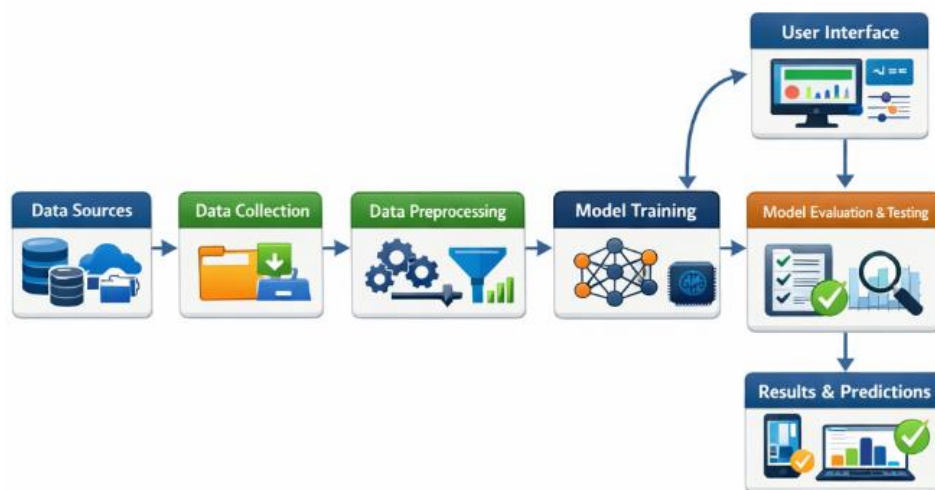


Fig. 4.2 Level 1: Detailed Diagram

4.3 UML Diagrams

4.3.1 Use Case Diagram

The Use Case Diagram for the system provides a visual representation of the interactions between the users (actors) and the system's functionalities. It highlights the primary actors, such as the Admin, Registered User, and Guest, and maps out the core use cases they can perform, including Login/Logout, Data Input, Prediction Generation, Report Viewing, Notification Alerts, and System Management. The diagram also illustrates relationships like include and extend to show dependent or optional functionalities, ensuring a clear understanding of how users engage with different modules. Overall, the use case diagram offers a high-level overview of the system's behavior and user interactions, serving as a foundational blueprint for both development and documentation.

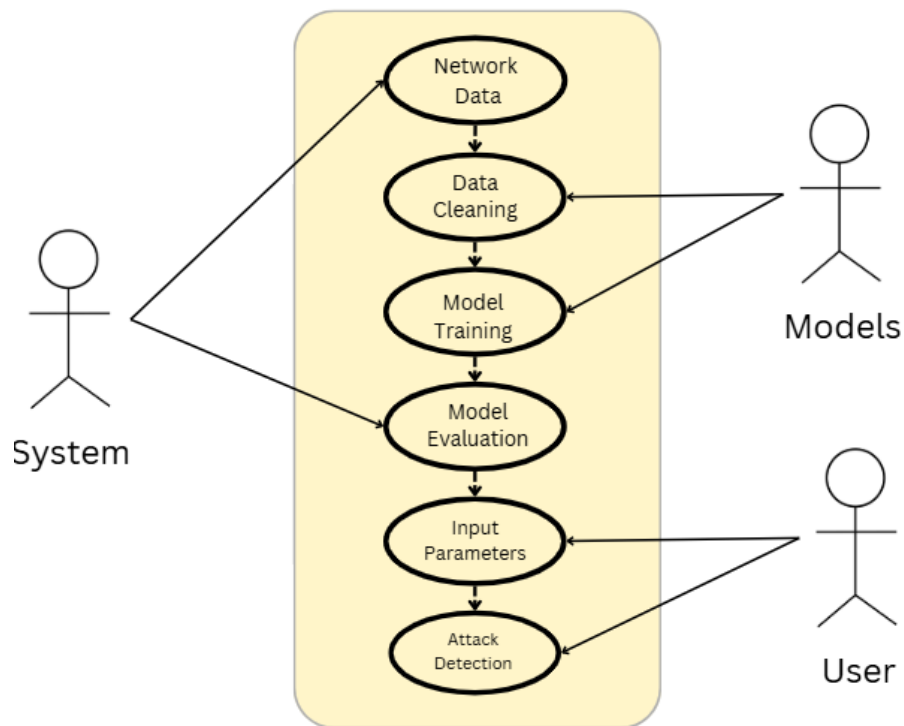


Fig. 4.3 Use Case Diagram

4.3.2 Sequence Diagram

The Sequence Diagram illustrates the dynamic interaction between the system's components and users over time, detailing the flow of messages for accomplishing specific tasks. It captures the step-by-step communication among actors such as Users, System Interface, Data Processing Module, Prediction Engine, and Database, showing how a user request, like submitting input data, triggers a sequence of operations including data validation, preprocessing, model invocation, result computation, and response generation. Lifelines and arrows represent the order of messages, while activation bars highlight the period each component is actively processing. By visualizing these interactions chronologically, the sequence diagram provides a clear understanding of the system's operational workflow, helping developers and stakeholders trace processes, detect bottlenecks, and ensure that each component functions cohesively within the overall system architecture.

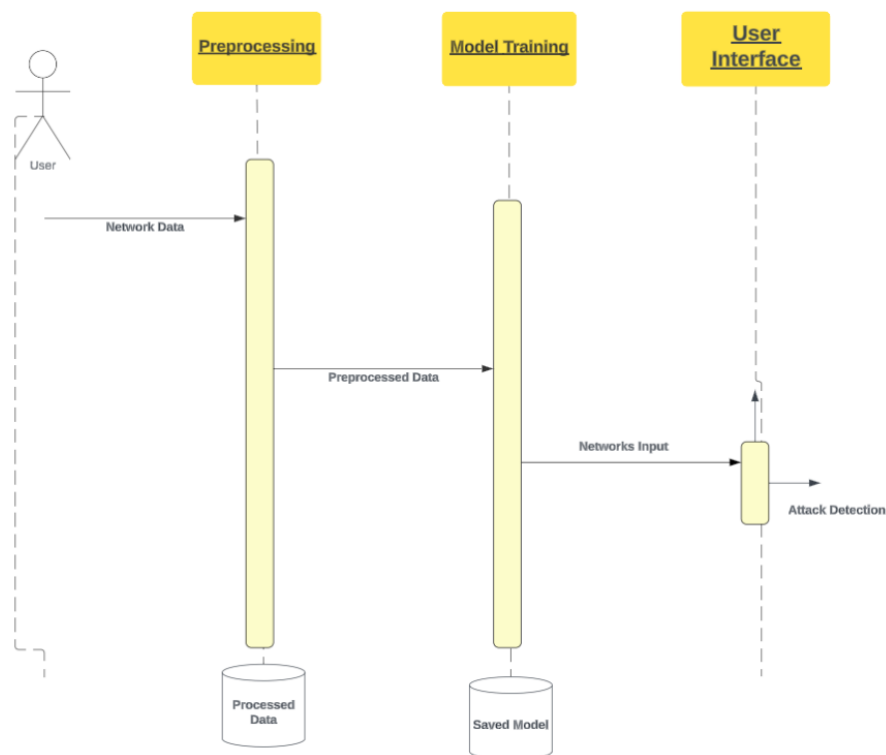


Fig 4.4 Sequence Diagram

4.3.3 Class Diagram

The Class Diagram provides a comprehensive and high-level view of the system's

overall structure by representing the main classes, their essential attributes, and the relationships among them in a unified model. It illustrates how core entities such as Users, Admins, Data Processing Modules, Prediction Engines, and Report or Notification Modules are organized and interconnected, highlighting the flow of information and responsibilities across the system. The diagram captures various types of relationships, including associations to show interactions between classes, inheritance to depict hierarchies or generalizations, and aggregations/compositions to indicate whole-part dependencies, giving a clear picture of how different components collaborate to achieve system functionality. By focusing on the structural organization rather than implementation details, the Class Diagram serves as a blueprint that clarifies the system's architecture, enhances understanding of component interactions, supports consistent design, and provides a foundation for development, scalability, and future maintenance, allowing stakeholders and developers to visualize the system's entities and their interconnections in a single cohesive representation.

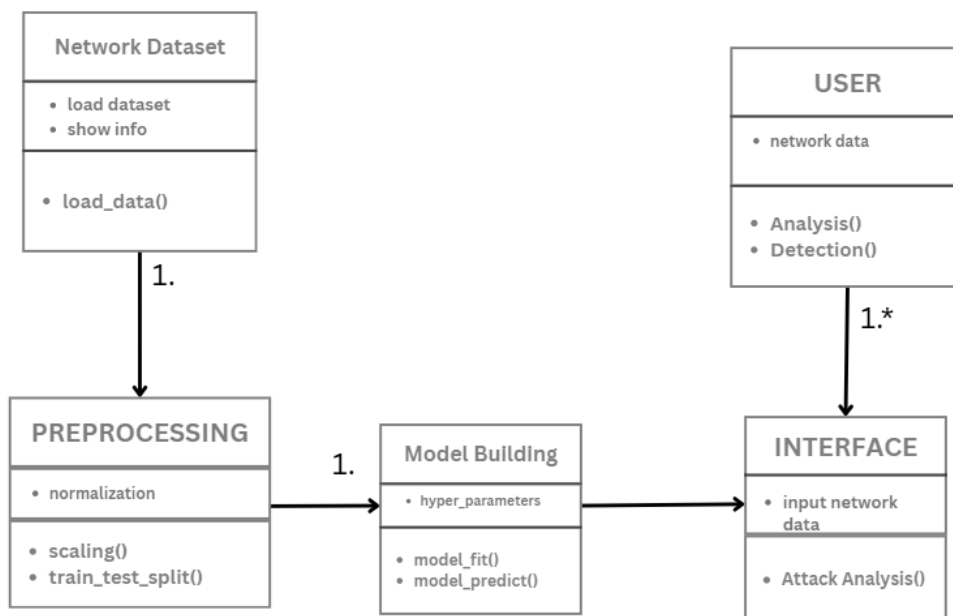


Fig.4.5 Class Diagram

4.3.4 Collaboration Diagram

The Collaboration Diagram provides a dynamic view of the system by illustrating how objects interact with one another to carry out specific tasks, emphasizing the relationships and message exchanges rather than the sequence of events. It shows objects such as Users, System Interface, Data Processing Modules, Prediction Engine, and Report/Notification Modules, and maps the flow of information between them through numbered messages that indicate the order of communication. Relationships like associations depict how objects are linked, while the message flow highlights responsibilities and collaborations necessary to complete operations, such as data submission, processing, prediction, and result delivery. By focusing on the structural organization of interactions, the Collaboration Diagram complements sequence diagrams by providing clarity on how different components cooperate, making it easier to understand object dependencies, system behavior, and overall workflow in a unified, interaction-focused perspective.

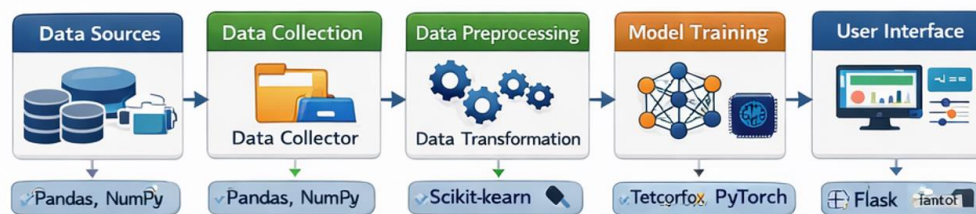


Fig.4.6 Collaboration Diagram

4.3.5 Activity Diagram

The Activity Diagram provides a visual representation of the workflow and operational

processes within the system, showing the sequence of activities, decision points, and parallel processes from start to finish. It illustrates how users or system components perform tasks such as data input, preprocessing, analysis or prediction, result generation, and reporting, with arrows indicating the flow of control between activities. Decision nodes, forks, and joins represent conditional branching and concurrent operations, while start and end points mark the beginning and completion of processes. By capturing the dynamic behavior of the system in a clear, step-by-step manner, the Activity Diagram helps stakeholders and developers understand the logic, dependencies, and flow of operations, ensuring that the process design is efficient, coherent, and easy to follow.

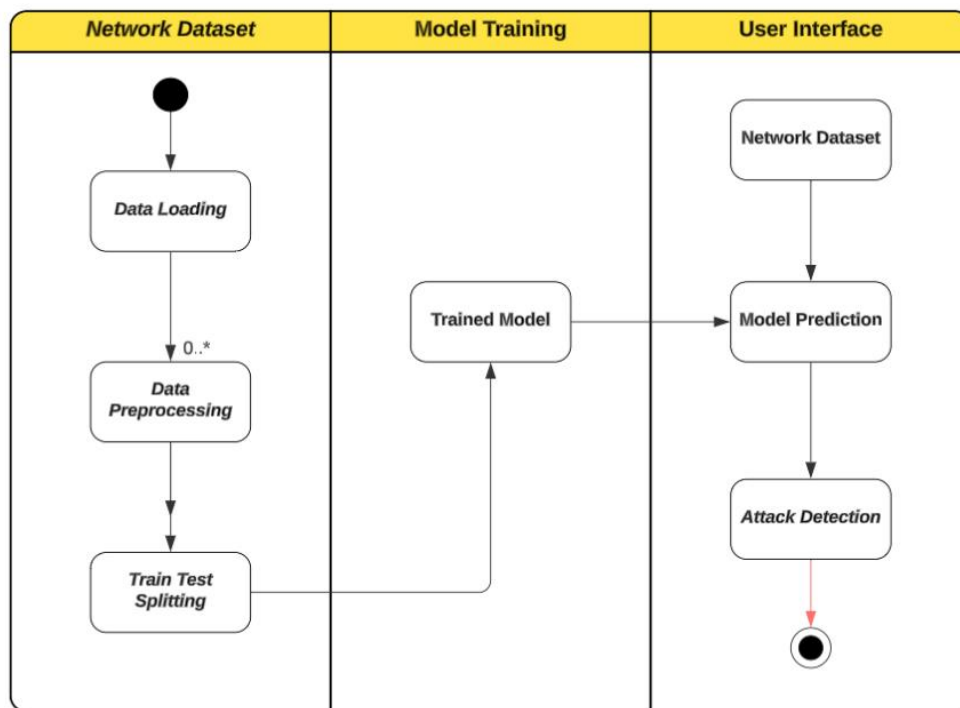


Fig.4.7 Activity Diagram

CHAPTER 5

System Coding & Testing **40-60**

- 5.1. Introduction
- 5.2. Implementation
- 5.3. About Python
- 5.4. Sample Code
- 5.5. Software Testing Techniques
- 5.6. Software Testing Strategies

5. SYSTEM CODING & TESTING

5.1 Introduction

The System Coding and Testing phase is a crucial stage in the development of the proposed DRL-based Intrusion Detection System, as it translates the system design into a fully functional and operational implementation. During this phase, all modules—including data collection and preprocessing, DRL agent training, evaluation, and user interface—are implemented using Python and integrated into a cohesive system. Coding involves developing scripts for dataset handling, feature extraction, model building with Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO), and interactive visualization of system outputs. Special attention is given to the design of the reward function, action space, and state representation to ensure that the reinforcement learning agent can learn optimal policies efficiently.

Proper coding practices, modular programming, and clear documentation are emphasized to enhance maintainability, readability, and scalability of the system. Testing plays an equally critical role, as it ensures that the implemented system meets the functional and performance requirements defined during system analysis and design. The testing phase involves unit testing for individual modules, integration testing to verify proper interaction between modules, and system-level testing to evaluate end-to-end functionality. The system is tested using the UNSW_NB15 dataset in both simulated and controlled environments, checking for accurate multi-class intrusion detection, minimal false alarms, and real-time response capabilities. Metrics such as detection accuracy, precision, recall, F1-score, and response time are monitored, while confusion matrices provide detailed insights into the system's ability to correctly classify different attack types. Iterative testing and debugging ensure that all modules function as expected, and any errors, inconsistencies, or performance bottlenecks are identified and resolved.

5.2 Implementation of the Project

The implementation of the proposed Deep Reinforcement Learning-based Intrusion Detection System begins with the preparation and preprocessing of network traffic data from the UNSW_NB15 dataset. Raw data is cleaned to handle missing values and inconsistencies, and relevant features such as packet size, protocol type, source and destination IPs, and flow-based statistics are extracted. Categorical variables are encoded, and numerical features are normalized to ensure consistent input for the reinforcement learning models. The processed dataset is then divided into training, validation, and testing sets, ensuring that the system can be effectively evaluated for multi-class attack detection and generalization across unseen network patterns. This step establishes a robust foundation for the DRL agent to learn from meaningful and representative data. The next stage involves training the DRL agent using advanced reinforcement learning algorithms, specifically Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO).

In this setup, network traffic features serve as the states, possible intrusion classifications are treated as actions, and a carefully designed reward function provides feedback to the agent by balancing detection accuracy, false alarm reduction, and response efficiency. The agent interacts with a simulated IoT network environment, updating its policy iteratively to optimize decision-making. Hyperparameter tuning is performed by systematically adjusting learning rates, batch sizes, and reward weights to achieve stable and efficient training. Techniques such as dropout and gradient clipping are incorporated to prevent overfitting and ensure smooth learning dynamics. The iterative training process continues until convergence is achieved, resulting in a robust DRL agent capable of accurately detecting multiple types of attacks in real-time.

5.3 About Python

Python is a high-level, interpreted programming language known for its simplicity,

readability, and versatility. It was created by Guido van Rossum and first released in 1991. Python uses a clean and easy-to-understand syntax, which makes it popular among beginners as well as experienced developers. It supports multiple programming paradigms such as object-oriented, procedural, and functional programming. Python is widely used in many fields including web development, data science, artificial intelligence, machine learning, automation, and scientific computing. It provides a large standard library and a rich ecosystem of frameworks and tools like Django, Flask, and TensorFlow that simplify development and enable developers to build powerful applications efficiently. Because of its flexibility, strong community support, and cross-platform compatibility, Python has become one of the most widely used programming languages in the world.

Features of Python:

- **Simple and Easy to Learn:** Python has a clear and readable syntax that is similar to the English language, making it easy for beginners to learn and understand programming concepts quickly.
- **Interpreted Language:** Python is an interpreted language, which means the code is executed line by line without the need for compilation. This makes debugging easier and speeds up development.
- **Object-Oriented and Multi-Paradigm:** Python supports object-oriented programming as well as other paradigms such as procedural and functional programming, allowing developers to build structured and reusable code.
- **Large Standard Library and Community Support:** Python provides a vast collection of built-in libraries and modules that help in tasks such as web development, data analysis, machine learning, and automation, along with strong global community support.

5.4 Sample Code

```
import streamlit as st

import pandas as pd

import numpy as np

import torch

from stable_baselines3 import PPO

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

# -----

# Load trained PPO model

# -----

ppo_model = PPO.load("Saved Model")


# Load scaler and feature columns (SAVE THESE DURING TRAINING!)

scaler = torch.load("scaler.pt")    # you must save scaler

feature_columns = torch.load("features.pt") # list(X.columns)


# -----

# Streamlit UI

# -----

st.set_page_config(page_title="DRL IDS PPO", layout="centered")


with st.container(border=True):

    st.markdown(

        """

        <h1 style='

            text-align: center;

            color: #1f4fd8;

            font-weight: 700;
```

```
margin-bottom: 10px;

'>

<img alt="Intrusion Detection System logo" data-bbox="222 151 242 171"/> Intrusion Detection System

</h1>

""",

unsafe_allow_html=True

)

# -----
# File Upload
# -----

uploaded_file = st.file_uploader("Upload Network Traffic Data", type=["csv"])

# -----
# Prediction function
# -----

def predict_single_row(model, row):
    obs = row.values.reshape(1, -1)
    obs_tensor = torch.tensor(obs).float()

    with torch.no_grad():
        dist = model.policy.get_distribution(obs_tensor)
        probs = dist.distribution.probs.cpu().numpy()[0]

    pred_class = np.argmax(probs)
    confidence = probs[pred_class] * 100

    return {
```

```

        "Prediction": "Attack" if pred_class == 1 else "Normal",
        "Normal Probability (%)": round(probs[0] * 100, 2),
        "Attack Probability (%)": round(probs[1] * 100, 2),
        "Confidence (%)": round(confidence, 2)
    }

# -----
# Main Logic
# -----

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)

    st.sidebar.subheader(" Uploaded Dataset Preview")
    st.sidebar.dataframe(df.head())

    #plot the distribution of the label column if it exists
    if 'label' in df.columns:
        st.sidebar.subheader(" Label Distribution")
        st.sidebar.bar_chart(df['label'].value_counts())

    # Drop optional columns safely
    df.drop(['id', 'attack_cat', 'label'], axis=1, inplace=True, errors='ignore')

    # One-hot encode
    df = pd.get_dummies(df)

# -----
# Dataset compatibility check
# -----

```

```
if not set(feature_columns).issubset(set(df.columns)):

    st.error(" ✖ Dataset Mismatch: Uploaded dataset does not match trained model features")

    st.stop()

# Align columns
df = df.reindex(columns=feature_columns, fill_value=0)

# Scale
df_scaled = scaler.transform(df)

# -----
# Single-row prediction
# -----
if df_scaled.shape[0] == 1:
    result = predict_single_row(
        ppo_model,
        pd.Series(df_scaled[0])
    )

    st.subheader("🔍 Prediction Result")
    st.json(result)

# -----
# Multiple-row prediction
# -----
else:
    col1,col2,col3 = st.columns([1,6,1])
```

```

row_index = st.slider(
    " Selected Row: {row_index}")
    with st.container(border=True):
        col1, col2 = st.columns([1, 1.5])

        # ----- COLUMN 1 -----

        with col1:
            st.markdown(
                f"""
                <div style="
                    background: rgba(0,0,0,0.1);

```



```
padding: 20px;
border-radius: 12px;
text-align: center;
color: white;
">

<h2><img alt="shield icon" data-bbox="368 238 385 255"/> Prediction</h2>

<h1 style="color: {'#ff4b4b' if result['Prediction']=='Attack' else
'#00c853'};">

    {result['Prediction']}

</h1>

<h3>Confidence</h3>

<h2 style="color: blue;">

    {result['Confidence (%)']}%

</h2>

</div>

""",

unsafe_allow_html=True

)

# ----- COLUMN 2 -----

with col2:

    labels = ["Normal", "Attack"]

    values = [

        result["Normal Probability (%)"],

        result["Attack Probability (%)"]

    ]

    colors = ["green", "red"]
```

```
fig, ax = plt.subplots()

ax.bar(labels, values, color=colors)

ax.set_ylim(0, 100)

ax.set_ylabel("Probability (%)")

ax.set_title("Intrusion Probability Distribution")


for i, v in enumerate(values):

    ax.text(i, v + 1, f"{v}%", ha="center", fontweight="bold")


st.pyplot(fig)
```

```
import numpy as np
```

```
import pandas as pd
```

```
import gym
```

```
from gym import spaces
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from stable_baselines3 import PPO, DDPG
```

```
from stable_baselines3.common.vec_env import DummyVecEnv
```

```
from stable_baselines3.common.noise import NormalActionNoise
```

```
df1 = pd.read_csv('Data/UNSW_NB15_training-set.csv')
```

```
df2 = pd.read_csv('Data/UNSW_NB15_testing-set.csv')
```

```
df = pd.concat([df1, df2], ignore_index=True)
```

```
# Separate features and label
```

```
X = df.iloc[:, :-1]
```

```
y = df.iloc[:, -1]
```

```
# Scale numeric features
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
print(X_train.shape, X_test.shape)
```

```
class IDSMDPEnv(gym.Env):
```

```
    def __init__(self, X, y):
```

```
        super(IDSMDPEnv, self).__init__()
```

```
        self.X = X
```

```
        self.y = y
```

```
        self.index = 0
```

```
        self.observation_space = spaces.Box(
```

```
            low=-np.inf,
```

```
            high=np.inf,
```

```
            shape=(X.shape[1],),
```

```
dtype=np.float32

)

self.action_space = spaces.Discrete(2)

def reset(self):
    self.index = 0
    return self.X[self.index]

def step(self, action):
    true_label = self.y.iloc[self.index]

    # Reward function
    if action == true_label:
        reward = 1.0
    elif action == 1 and true_label == 0:
        reward = -1.0 # False Positive
    else:
        reward = -2.0 # False Negative

    self.index += 1
    done = self.index >= len(self.X) - 1

    next_state = self.X[self.index] if not done else np.zeros(self.X.shape[1])

    return next_state, reward, done, {}
```

```
class IDSMDPEnvDDPG(IDSMDPEnv):  
    def __init__(self, X, y):  
        super().__init__(X, y)  
        self.action_space = spaces.Box(low=0.0, high=1.0, shape=(1,))  
  
    def step(self, action):  
        action = int(action[0] > 0.5)  
        return super().step(action)  
  
ppo_model = PPO(  
    policy="MlpPolicy",  
    env=env,  
    verbose=1,  
    learning_rate=3e-4,  
    gamma=0.99,  
    batch_size=256,  
    n_steps=2048  
)  
  
ppo_model.learn(total_timesteps=200_000)  
  
ppo_model.save("Saved Model")  
  
def evaluate(model, X, y, is_ddpg=False):  
    env = IDSMDPEnvDDPG(X, y) if is_ddpg else IDSMDPEnv(X, y)  
    obs = env.reset()  
  
    correct = 0
```

```

for i in range(len(X)-1):
    action, _ = model.predict(obs, deterministic=True)
    if is_ddpg:
        action = int(action[0] > 0.5)
    correct += int(action == y.iloc[i])
    obs, _, done, _ = env.step(action)
    if done:
        break

return correct / len(X)

```

```

def get_predictions(model, X, y):
    env = IDSMDPEnv(X, y)
    obs = env.reset()
    y_true, y_pred, y_prob = [], [], []
    for i in range(len(X)-1):
        action, prob = model.predict(obs, deterministic=True)
        y_true.append(y.iloc[i])
        y_pred.append(action)
        # PPO does not output probability → approximate
        y_prob.append(float(action))
        obs, _, done, _ = env.step(action)
        if done:
            break

    return np.array(y_true), np.array(y_pred), np.array(y_prob)

```

```

y_true, y_pred, y_prob = get_predictions(ppo_model, X_test, y_test)

```

```
acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc = roc_auc_score(y_true, y_prob)

print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall : {rec:.4f}")
print(f"F1-score : {f1:.4f}")
print(f"ROC-AUC : {auc:.4f}")

import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Attack'],
            yticklabels=['Normal', 'Attack'])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix – PPO IDS")
plt.show()

fpr, tpr, _ = roc_curve(y_true, y_prob)
```

```
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}')
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve – PPO IDS')
plt.legend()
plt.grid()
plt.show()

from sklearn.metrics import precision_recall_curve
precision, recall, _ = precision_recall_curve(y_true, y_prob)
plt.figure(figsize=(6,5))
plt.plot(recall, precision)
plt.xlabel('Recall')
plt.ylabel('Precision')
```

5.5 Software Testing Techniques Used

1. Unit Testing

Unit testing focuses on verifying the functionality of individual components or modules of the system in isolation. It ensures that each unit, such as a function, class, or method, performs as intended and produces the expected output for given inputs. Developers often write test cases for every possible scenario, including normal, boundary, and error conditions, to identify and fix defects at an early stage. By testing units independently before integrating them into the complete system, unit testing helps reduce downstream errors, simplifies debugging, and improves code quality.

2. Integration Testing

Integration testing evaluates how different modules or components of the system work together when combined. While unit testing ensures individual functionality, integration testing focuses on detecting interface defects, data flow issues, and interaction problems between modules. It can be performed using approaches such as top-down, bottom-up, or big bang integration, depending on the project structure. This type of testing ensures that combined modules cooperate seamlessly, data is passed correctly between them, and the overall system behaves as expected.

3. System Testing

System testing is a comprehensive evaluation of the fully integrated software to ensure that it meets all functional and non-functional requirements. Unlike unit or integration testing, system testing treats the application as a complete entity and validates its end-to-end behavior in an environment that simulates real-world usage. It involves checking for functionality, performance, reliability, usability, and security, ensuring that the system behaves as intended under various scenarios. This phase also identifies any discrepancies between the design specifications and the implemented system, providing a final verification before deployment. System testing is critical for ensuring user satisfaction, minimizing post-deployment defects, and confirming that the software fulfills its intended purpose under actual operating conditions.

4. Acceptance Testing

Acceptance testing is performed to determine whether the system satisfies the end-users' requirements and is ready for deployment. Typically conducted by the client or end-users, it focuses on validating real-world functionality, usability, and compliance with business needs. Acceptance testing can include alpha testing, conducted within the organization before release, and beta testing, carried out with a select group of external users to gather feedback on performance and usability. This type of testing ensures that the software not only works technically but also meets practical expectations and operational standards. By identifying any remaining issues from the user's perspective, acceptance testing serves as the final validation before the system goes live.

5. Regression Testing

Regression testing is performed to ensure that recent changes, such as bug fixes, enhancements, or updates, do not adversely affect the existing functionality of the system. It involves re-executing previously run test cases and comparing results to ensure consistency and stability. Regression testing can be automated to handle repetitive tests efficiently, allowing rapid verification of system integrity after code modifications. This testing is essential for maintaining software quality over time, as it detects unintended side effects caused by new changes, ensuring that the system continues to operate correctly and reliably even as it evolves.

6. Performance Testing

Performance testing evaluates how the system behaves under various conditions related to speed, responsiveness, stability, and scalability. It focuses on measuring critical metrics such as load handling, response time, throughput, and resource utilization, identifying bottlenecks or inefficiencies that could impact user experience. Common types of performance testing include load testing, stress testing, and scalability testing, each designed to assess the system under specific scenarios.

5.6 Software Testing Strategies Used

1. Top-Down Testing Strategy

The Top-Down Testing Strategy is an incremental approach where testing begins with the top-level modules or main control functions of the system and progressively moves downward to the subordinate modules. In this strategy, high-level modules are tested first using stubs to simulate the lower-level modules that are not yet developed or integrated. This approach ensures that the main control and major functionality are validated early, providing a clear view of the system's overall structure and behavior. Top-down testing helps identify design issues, interface errors, and control flow problems at an early stage, which reduces the effort and cost of fixing defects later.

2. Bottom-Up Testing Strategy

The Bottom-Up Testing Strategy starts by testing the lower-level, detailed modules first, and gradually integrates them into higher-level modules until the complete system is tested. Drivers are often used to simulate higher-level modules that are not yet developed, enabling lower-level modules to be tested independently. This approach ensures that foundational components are stable and function correctly before integrating them into more complex modules. Bottom-up testing helps detect defects in individual modules early, improves the reliability of component interactions, and ensures that each building block performs correctly, forming a solid base for higher-level system integration.

3. Big Bang Testing Strategy

The Big Bang Testing Strategy involves integrating all modules simultaneously after development and then testing the system as a whole. Unlike incremental approaches, Big Bang testing does not test modules individually before integration, which means all functionalities are verified together. This strategy allows the system to be evaluated in its complete form, revealing interface issues, integration defects, and overall system behavior under realistic conditions. While it can be effective for small systems with few modules, it may be challenging to identify the origin of defects in large or complex systems due to the simultaneous integration of all components.

4. Functional Testing Strategy

The Functional Testing Strategy focuses on verifying that the system performs

according to its specified requirements and functional specifications. It examines individual features, operations, and outputs without concern for internal code structure, emphasizing black-box testing techniques. Test cases are designed to cover all input scenarios, boundary conditions, and expected results to ensure that the system meets business and user requirements. Functional testing helps validate that each function works correctly, detects missing or incorrect functionality, and ensures compliance with user expectations.

5. Structural (White-Box) Testing Strategy

The Structural or White-Box Testing Strategy focuses on examining the internal logic, code structure, and pathways of the system to ensure that every part functions as intended. It involves testing loops, branches, conditions, and internal paths to detect errors that may not be visible in functional testing. Techniques such as path coverage, statement coverage, and branch coverage are commonly used to measure the completeness of the tests.

6. User Acceptance Testing (UAT) Strategy

The User Acceptance Testing (UAT) Strategy is conducted to confirm that the system satisfies the real-world needs of its end-users and stakeholders before deployment. In this strategy, actual users test the system in scenarios that simulate normal operational use to validate functionality, usability, and performance from a practical perspective. UAT focuses on ensuring that the system aligns with business requirements, user expectations, and workflow processes, identifying any gaps, usability issues, or missing features. By involving end-users in testing, this strategy provides critical feedback for improvements, builds user confidence, and ensures that the system is ready for live implementation, ultimately bridging the gap between technical verification and practical applicability.

CHAPTER 6

RESULTS

61-66

6.1. Introduction

6.2. Output Screens

6. RESULTS

6.1 Introduction

The results section of the proposed DRL-based Intrusion Detection System provides a comprehensive analysis of the system's performance in detecting and classifying multi-class attacks within IoT networks. This section focuses on evaluating the effectiveness, accuracy, and robustness of the trained reinforcement learning models—Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO)—using the UNSW_NB15 dataset. The evaluation considers multiple performance metrics, including detection accuracy, precision, recall, F1-score, false alarm rate, and response time. By analyzing these metrics, the study demonstrates how the system adapts to evolving network traffic patterns, identifies previously unseen attacks, and maintains a balance between accurate detection and minimal false alerts, which is critical in dynamic IoT environments.

In addition to performance metrics, the results analysis includes visualizations such as confusion matrices, accuracy trends, and attack-wise detection rates, providing detailed insights into the system's multi-class classification capabilities. These visualizations allow for a clear understanding of how each attack type is detected, highlighting strengths and potential areas for improvement. Comparative evaluation against conventional static intrusion detection systems and classical machine learning approaches emphasizes the advantages of the proposed DRL-IDS, particularly in terms of adaptability, real-time response, and consistent high detection performance. This analysis validates the system's design and implementation choices, including the use of reinforcement learning for continuous policy improvement, the reward-based decision-making mechanism, and preprocessing techniques that optimize input data quality for the agent.

6.2 Output Screens

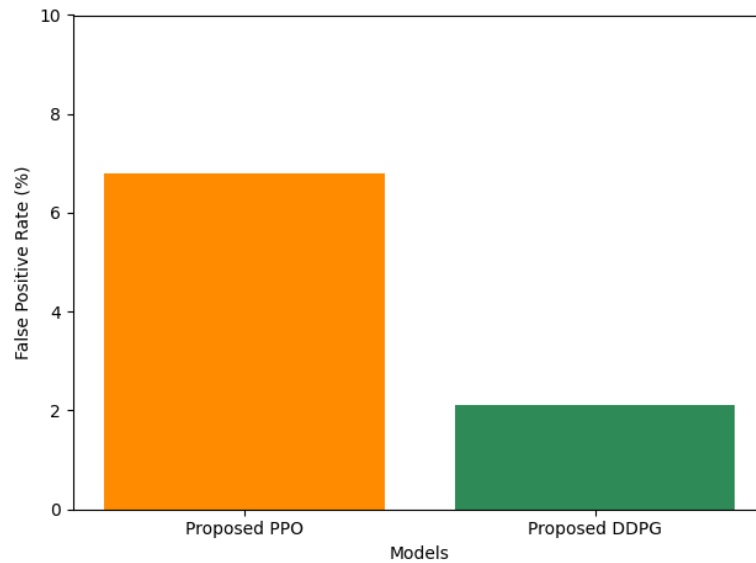


Fig.6.1 False Positive Rate Comparison of Proposed Models

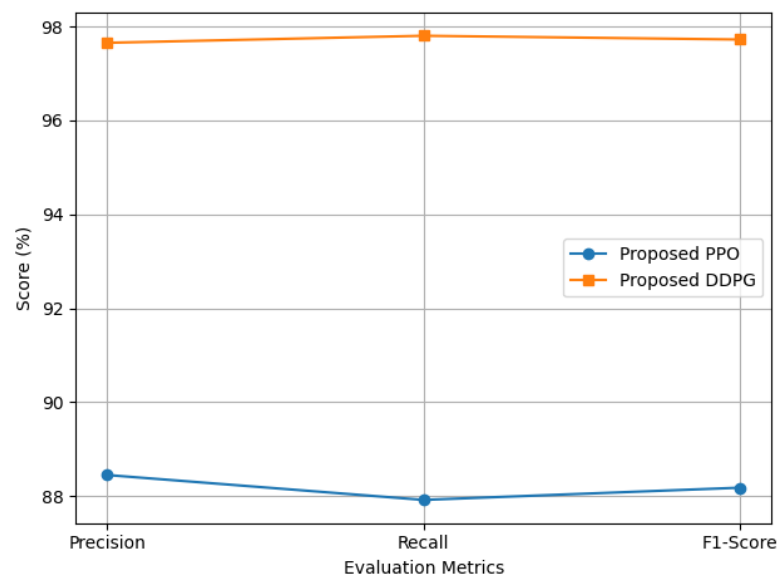


Fig 6.2 Performance Metrics of Proposed DRL-Based IDS

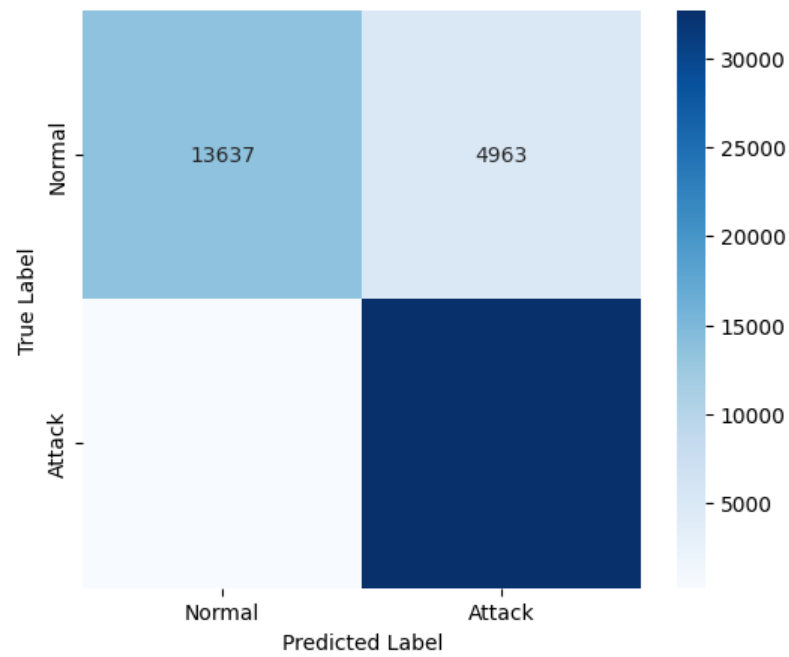


Fig.6.3 Confusion Matrix of Proposed Model

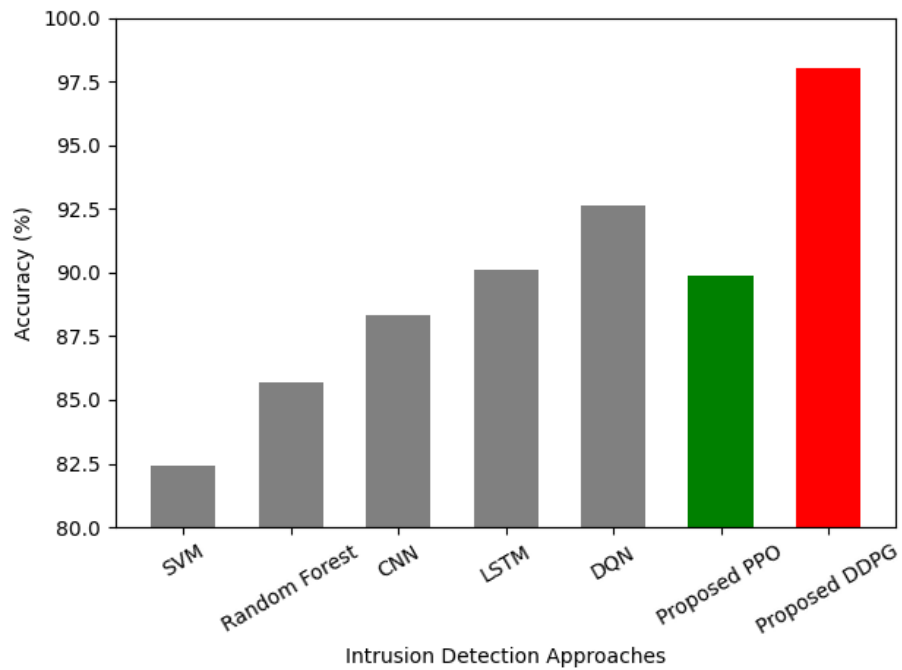


Fig 6.4 Existing vs Proposed IDS Accuracy Comparison

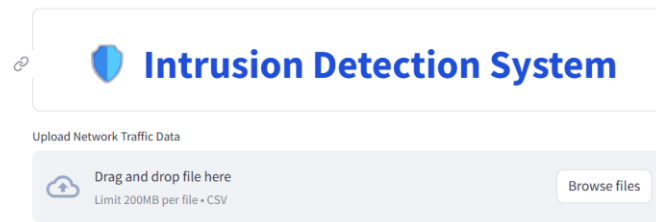


Fig 6.5 Home Page

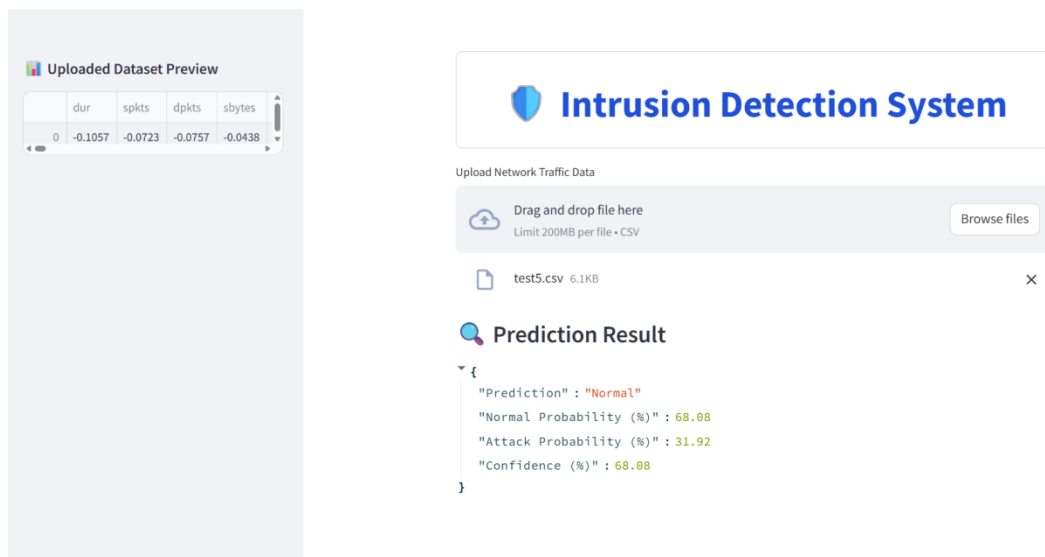


Fig 6.6 Results Page

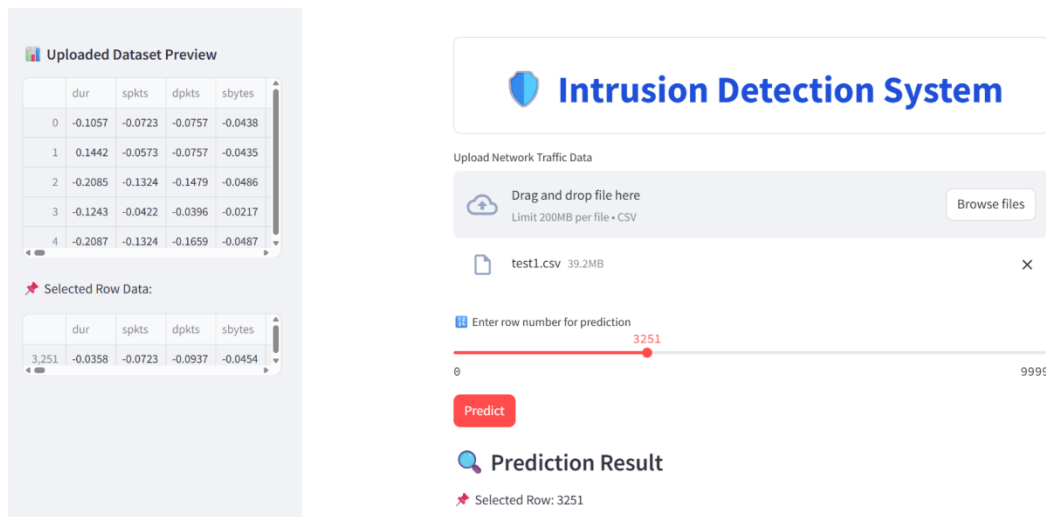


Fig 6.7 Results Page

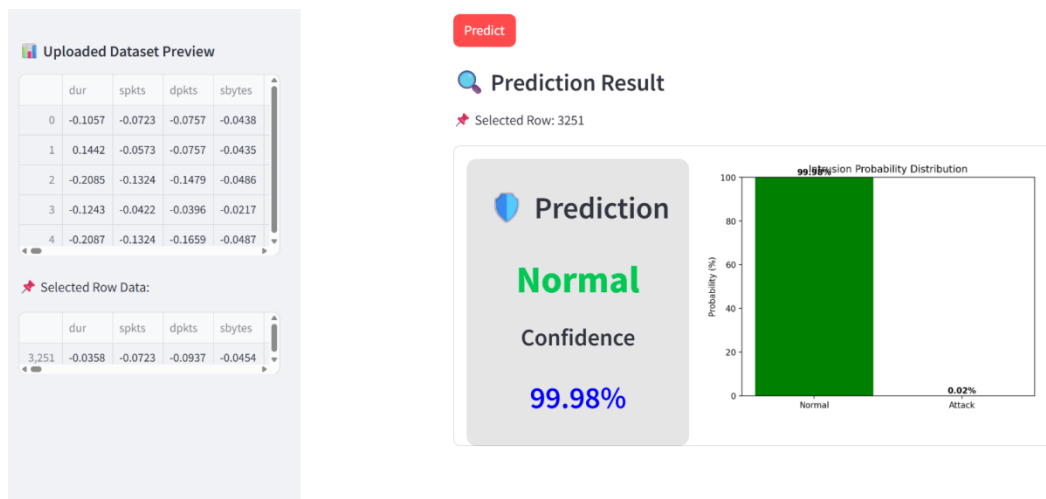


Fig 6.8 Results Page

CHAPTER 7

Conclusion & Future Enhancements 67-69

7.1. Conclusion

7.2. Future Enhancements

7. CONCLUSION & FUTURE ENHANCEMENTS

7.1. Conclusion

The proposed Deep Reinforcement Learning-based Intrusion Detection System effectively addresses the limitations of traditional and static intrusion detection approaches in modern IoT networks. By formulating the intrusion detection task as a Markov Decision Process, the system leverages reinforcement learning algorithms, specifically Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO), to enable adaptive and intelligent decision-making. Through continuous interaction with a simulated network environment using the UNSW_NB15 dataset, the DRL agent learns optimal detection policies that balance accuracy, false alarm reduction, and response efficiency, demonstrating superior performance compared to conventional methods. This adaptive capability ensures that the system remains robust against evolving and sophisticated cyber threats, which is critical in heterogeneous and dynamic IoT ecosystems.

The implementation and evaluation of the system highlight its effectiveness in multi-class attack detection, including DDoS, ransomware, backdoor, injection, and man-in-the-middle attacks. Extensive preprocessing, hyperparameter tuning, and model optimization contribute to stable and reproducible results, with the DDPG model achieving an impressive average accuracy of 98 percent, while the PPO model attains 89.9 percent. Performance metrics such as precision, recall, F1-score, and confusion matrices further validate the system's ability to accurately classify attacks while minimizing false positives. These results confirm that the DRL-based approach not only improves detection accuracy but also provides actionable insights and real-time alerts, making it a reliable and practical solution for IoT network security.

7.2 Future Enhancements

Future enhancements for the proposed DRL-based Intrusion Detection System can focus on improving scalability and real-time performance in larger and more complex IoT networks. While the current system performs effectively on the UNSW_NB15 dataset and a simulated environment, real-world IoT deployments often involve a massive number of heterogeneous devices generating continuous traffic. Enhancing the system to handle high-throughput networks and distributed IoT environments will require optimization techniques such as model pruning, quantization, and edge-based deployment strategies. Additionally, integrating parallel processing and GPU acceleration can further reduce response times and enable near real-time intrusion detection, which is critical for timely mitigation of sophisticated attacks.

Another potential enhancement is the incorporation of federated reinforcement learning to allow multiple IoT devices or edge nodes to collaboratively train and update the DRL agent without centralizing sensitive data. This approach would not only enhance privacy and security but also enable the system to learn from diverse network conditions and attack patterns across different devices. By distributing the learning process, the system can adapt to localized threats while maintaining global detection intelligence. Moreover, incorporating online learning mechanisms can help the agent continuously update its policy in response to newly emerging attack types, further improving adaptability and robustness. Finally, future work can focus on advanced threat intelligence integration and multi-modal data analysis. Incorporating additional sources of information, such as system logs, device behavior profiles, and environmental sensors, can enhance the context-awareness of the DRL agent and improve detection accuracy. Additionally, combining reinforcement learning with other AI approaches, such as graph-based anomaly detection or attention-based deep learning models, may improve the system's capability to identify complex, coordinated attacks like botnets or advanced persistent threats.